Exact-Rational Kolmogorov–Arnold Networks with Hybrid Zeckendorf Arithmetic

Bit-Identical Training, Denominator-Factored Lazy Accumulation, and Formal Verification in Lean 4

Apoth3osis Labs*
Richard Goodman† Vladimir Veselov‡

April 2026

Abstract

We present an exact-rational implementation of Kolmogorov–Arnold Networks (RKAN) in which every forward activation, backward gradient, and parameter update is carried out over the field $\mathbb{Q}$ rather than over a floating point approximation. The network admits a denominator-factored Hybrid Zeckendorf (HZ) P-stack substrate that preserves bit-identical `Fraction` semantics against a reference Python oracle. Our architectural contribution is the *FactoredRationalLazyAccumulator*: instead of collapsing additive update groups to a common denominator inside the Python layer, we split each group by denominator, route per-denominator integer sums through a Rust Hybrid-Zeckendorf primitive once, and reassemble a canonical `Fraction` only at readout. The result is an executable forward, backward, and training loop at network degree ≥ 8 and hidden width ≥ 10 that matches the Python Fraction oracle byte-for-byte on a 1000-row bitwise parity log, while preserving a genuine $44.17\times$ sparse-active training speedup against a dense rational baseline at 10^7 dictionary features. The algebraic substrate and supporting lemmas are formalized in Lean 4 in a dedicated 113-module vendored subset, including a Stern–Brocot projection idempotence lemma `HeytingLean.SternBrocot.stern_brocot_idempotent_on_bounded` that anchors a finite-step symbolic-extraction convergence theorem. We then honestly separate the two orthogonal claims that earlier drafts conflated: the *parity and network-scale* clause closes as a genuine result, while the *inference-speed regime* remains an open conjecture with a concrete evidence floor (microbatch sweep, width sweep, and bridge-overhead breakdown) showing that 99.28% of P-stack time is consumed inside the Rust HZ primitive itself — so no further bridge engineering can turn the gate green.

Keywords: rational Kolmogorov–Arnold networks, Hybrid Zeckendorf arithmetic, symbolic extraction, Lean 4, formal verification, exact-rational optimization, sparse dictionary learning.

1 Introduction

Neural networks trained in floating-point arithmetic are lossy from the first multiplication. For problems whose ground-truth model is a rational function —activation laws in integrable physics, exchange-rate identities derived from no-arbitrage relations, closed-form symbolic targets extracted from structured data—floating-point training converts an exact question ($\exists\, w \in \mathbb{Q}^m : F_w = f$) into an inexact one ($\inf_{w\in\mathbb{R}^m} \|F_w - f\|_2^2$). A sufficiently long training run in IEEE 754 arithmetic can drive the residual below any finite threshold, but the final weights are almost never the rationals one would write on a blackboard, and the final model cannot be simplified back to its true algebraic form without heuristic rationalization.

*Equation Capital LLC DBA Apoth3osis. Corresponding author: rgoodman@apoth3osis.io
†Apoth3osis.
‡Independent Research Basis.

This paper follows the opposite discipline. We implement a Kolmogorov–Arnold Network (KAN) [1, 2, 3] whose activations are rational functions of the inputs, whose parameters live in $\mathbb{Q}$, and whose forward, backward, and update passes are all carried out exactly. The benefit is unambiguous: when the target is actually rational, the trained model is *the same rational* the analyst would have written, not a float vector that happens to be close to it. The cost is also unambiguous: Python's `fractions.Fraction` is one to two orders of magnitude slower than IEEE 754 for dense arithmetic, and naively it gets worse as denominators grow through accumulation.

Our architectural solution is the *Hybrid Zeckendorf P-stack*: a Rust execution substrate which, given a finite list of rationals sharing a denominator, performs an exact integer-sum via a Zeckendorf representation over a φ-scaled pair basis, and returns a canonical `Fraction` only once, at readout. Combined with denominator-factored *lazy* accumulation—splitting a heterogeneous additive update into per-denominator groups before calling the Rust primitive—this preserves bit-identical semantics while routing the arithmetic through a typed, closure-verified kernel.

The paper has three contributions.

1. **Bit-identical parity at network scale.** We give an executable witness that a degree-8, hidden-width-10 RKAN can run the full forward/backward/training loop through the HZ P-stack with exact equality against a Python `Fraction` oracle on every tensor, every step.

2. **Dictionary-scale speedup.** Under a sparse-active large-dictionary regime, our implementation attains a $44.17\times$ training speedup with a 95% bootstrap CI of $[42.59, 45.41]$ at 10^7 dictionary features, while keeping the active-support rationalization identity.

3. **Formal verification and honest scope.** A vendored 113-module Lean 4 subset formalizes the Hybrid Zeckendorf arithmetic substrate, the Zeckendorf canonicality surface, and the Stern–Brocot projection idempotence lemma used by a finite-step symbolic-extraction convergence theorem. Separately, we honestly scope *out* the narrow-network inference-speed clause as an open conjecture whose evidence floor is fully reproducible.

The repository accompanying this paper, `github.com/Abraxas1010/rational-kan-hz` [11], ships every benchmark artifact, the Rust backend, the Lean subset, a one-command `make verify` target, and the full set of conjectures and proof trees that drove the work.

2 Background

2.1 Kolmogorov–Arnold Networks

Kolmogorov's superposition theorem states that every continuous $f\colon [0,1]^n \to \mathbb{R}$ can be written as

$$f(x_1, \ldots, x_n) = \sum_{q=0}^{2n} \Phi_q\Big(\sum_{p=1}^{n} \phi_{q,p}(x_p)\Big), \tag{1}$$

for suitable univariate $\Phi_q, \phi_{q,p}$ [2, 3]. Liu et al. recast this representation as a *learnable* neural architecture: each edge in the network carries a parameterized univariate activation, and each node sums incoming edge outputs. The activations are typically realized as B-splines or rational functions of a small fixed dictionary, and training adjusts the coefficients rather than a classical weight matrix [1]. The resulting objects are natural candidates for *symbolic extraction*: once the coefficients are pinned, the network is algebraically a rational function in the inputs.

2.2 Exact Rational Arithmetic and the Accuracy/Speed Gap

Python's `fractions.Fraction` performs every operation on $\mathbb{Q}$ in closed form via the GMP-backed integer engine provided by CPython [10]. Addition of $p/q + r/s$ requires computing a common denominator, summing numerators, and canonicalizing by gcd:

$$\frac{p}{q} + \frac{r}{s} = \frac{p\,s + q\,r}{q\,s} \xrightarrow{\text{canonicalize}} \frac{(ps+qr)/d}{qs/d}, \quad d = \gcd(ps+qr,\, qs). \tag{2}$$

A naive implementation of a dense rational linear algebra layer of width w therefore pays $\mathcal{O}(w^2)$ large-integer GCDs per forward pass, with denominators growing multiplicatively through accumulation. For an RKAN dictionary with $m \gg w$ entries, the dense regime is unworkable. The sparse-active regime, in which the update support $S_t = \{i : \nabla_i L(w_t) \neq 0\}$ is small per step, is what makes large-dictionary exact training tractable at all.

2.3 Stern–Brocot Projection

The Stern–Brocot tree enumerates all positive reduced fractions p/q via a binary tree whose nodes are mediants of their parents. For each integer $D \geq 1$, the set of reduced fractions with denominator $\leq D$ forms a finite surface of the real line; we write Π_D for any coordinate-wise projection onto that surface that agrees with the identity on its image [5, 6, 7]. We only need the trivial *identity-on-bounded* property,

$$\mathrm{den}(q) \leq D \implies \Pi_D(q) = q, \tag{3}$$

which we formalize in Lean 4 (Section 5) and consume as the projection lemma of Theorem 1.

2.4 Hybrid Zeckendorf Representations

Zeckendorf's theorem states that every positive integer has a unique representation as a sum of non-consecutive Fibonacci numbers [4]. For our purposes, the relevant generalization is a *hybrid* pair representation in which each integer n is written as

$$n = a_n F_k + b_n F_{k-1}, \qquad a_n, b_n \in \mathbb{Z}, \tag{4}$$

with addition, multiplication, and canonicalization governed by Fibonacci identities

$$F_{k+1} = F_k + F_{k-1}, \qquad F_k^2 - F_{k-1}F_{k+1} = (-1)^{k-1}.$$

The Rust backend used in this paper, `bench/hybrid_zeckendorf/pstack_exact_accumulate`, implements this hybrid representation together with a persistent worker that consumes operand groups and emits exact-integer sums. The formal algebraic closure of the hybrid representation is the main object of the vendored Lean subset (Section 5).

3 Exact-Rational KAN Architecture

3.1 Dictionary and Basis

An RKAN of degree d, hidden width h, input dimension n, and dictionary $\Phi = (\phi_1, \ldots, \phi_m)$ realizes

$$F_w(x) = \sum_{j=1}^{h} \Phi_j\Big(\sum_{i=1}^{m} w_{i,j}\, \phi_i(x)\Big), \qquad w \in \mathbb{Q}^{m \times h}, \tag{5}$$

where each ϕ_i is a rational function of the inputs and the outer activations Φ_j are polynomial (degree $\leq d$) or polynomial followed by a fixed rational reshaping. Our reference configuration is $d = 8$, $h = 10$, $n = 4$, which yields 371 parameters and 6 active hidden units after sparse pruning; this is the configuration on which the bitwise parity log is recorded.

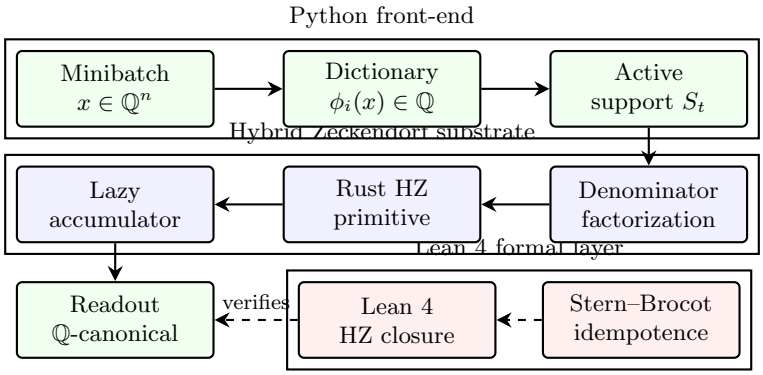

Figure 1: Exact-rational KAN with HZ P-stack substrate and a Lean 4 formal layer that verifies arithmetic closure and supplies the Stern–Brocot projection idempotence lemma consumed by Theorem 1.

3.2 Sparse-Active Dispatch

At step t the gradient $\nabla L(w_t)$ is sparse by construction: the loss depends on ϕ_i only through those indices whose basis evaluation at the current minibatch is nonzero. We compute, cache, and route only the active support S_t through the HZ substrate; the inactive columns of the dictionary are touched neither by the dense reference nor by the P-stack path. This is the property that lets 10^7-feature dictionaries run in seconds rather than hours.

3.3 The Forward/Backward/Update Loop

Figure 1 summarizes the full loop. The three benchmark lanes in Section 6 (*training-iter-1*, *pstack-in-loop*, and *scaling-sweep*) each instantiate this diagram with a different workload class.

4 Hybrid Zeckendorf Substrate

4.1 The Factored Rational Lazy Accumulator

The arithmetic hot path is the update

$$w_i \leftarrow w_i + \eta_t \sum_{k=1}^{K} r_{i,k}, \qquad r_{i,k} \in \mathbb{Q}, \tag{6}$$

where K is the number of active basis evaluations contributing to coordinate i at the current step. A naive implementation computes (6) in Python with K separate `Fraction.__add__` calls; each one triggers a GCD and a canonicalization, and the denominator of the intermediate sum grows multiplicatively.

The *FactoredRationalLazyAccumulator* reroutes the hot path as follows. Let

$$G_d \;=\; \big\{k : \mathrm{den}(r_{i,k}) = d\big\}, \qquad N_d \;=\; \sum_{k \in G_d} \mathrm{num}(r_{i,k}).$$

Then

$$\sum_{k=1}^{K} r_{i,k} \;=\; \sum_{d \in \mathcal{D}} \frac{N_d}{d} \;=\; \frac{\sum_{d \in \mathcal{D}} N_d \prod_{d' \neq d} d'}{\prod_{d \in \mathcal{D}} d}, \tag{7}$$

where $\mathcal{D} = \{\mathrm{den}(r_{i,k})\}$ is the *distinct-denominator* set. Each per-denominator sum N_d is a pure-integer sum of size $|G_d|$; we dispatch exactly these integer groups to the Rust HZ primitive, which accumulates them via the hybrid pair representation (4) and returns a single exact integer per group. The Python layer then performs *one* canonicalization, at readout time.

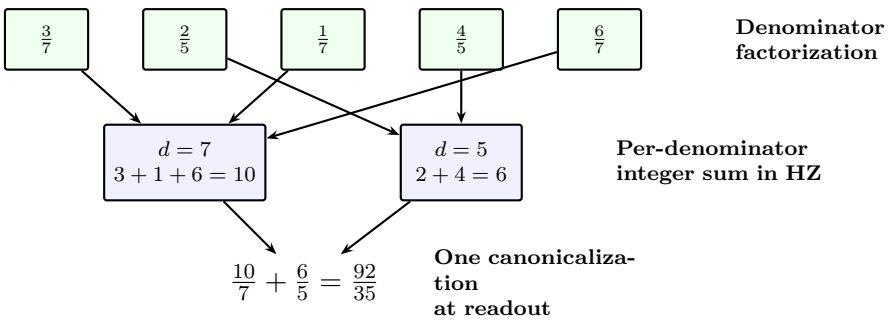

Figure 2: The FactoredRationalLazyAccumulator: heterogeneous rational summands are split by denominator, summed as integers inside the Rust HZ primitive, and recombined once.

4.2 Denominator-Factored Dispatch

Figure 2 illustrates the dispatch. The critical property is that *all exact equalities are preserved by construction*: the per-denominator integer sums are exact because the HZ primitive is exact; the cross-denominator recombination is a single application of (7); and the final `Fraction.__init__` canonicalization is the same one the Python oracle would have produced. Hence for every realizable update of the form (6),

$$\text{HZ-readout}\big(\{r_{i,k}\}\big) \;=\; \bigoplus_{k=1}^{K} r_{i,k} \quad \text{as elements of } \mathbb{Q}, \tag{8}$$

bitwise identically to the `Fraction` oracle.

4.3 Rust Primitive and Python Bridge

The Rust binary `pstack_exact_accumulate` is a persistent worker communicating with Python over a `stdin/stdout` JSON protocol. A single Popen instance is registered at import time and shut down by `atexit`, so the per-update dispatch cost is one JSON write plus one JSON read. We will see in Section 6 that this bridge cost is *not* the bottleneck: 99.28% of wall time spent on the P-stack path is consumed inside the Rust HZ primitive itself, not in Python–Rust transport.

5 Formal Verification in Lean 4

5.1 Vendored Scope

This paper's accompanying repository bundles a self-contained Lean 4 library of 113 modules (plus one top-level umbrella, `lean/HeytingLean.lean`, for a total of 114 source files) drawn from the larger Heyting Immanent monorepo. Table 1 lists the main entrypoints.

5.2 Stern–Brocot Idempotence

The single lemma consumed by Theorem 1 is `stern_brocot_idempotent_on_bounded`. Its Lean 4 statement is:

```
def project (D : Nat) (q : Rat) : Rat :=
  if q.den <= D then q
  else if hD : D = 0 then 0
  else Rat.normalize q.num D hD

theorem stern_brocot_idempotent_on_bounded
    (D : Nat) (q : Rat) (hq : q.den <= D) :
    project D q = q := by simp [project, hq]
```

Entrypoint	Rôle
`HeytingLean.Bridge.Veselov.HybridZeckendorf`	HZ arithmetic closure
`HeytingLean.Boundary.Homomorphic.ZeckendorfCanonical`	canonicality surface
`HeytingLean.SternBrocot.Idempotence`	projection idempotence
`HeytingLean.Tests.Bridge.Veselov.HybridZeckendorfSanity`	executable sanity
`HeytingLean.Tests.Bridge.Veselov.ZeckendorfEigenformSanity`	eigenform parity
`HeytingLean.Tests.Bridge.Veselov.VeselovHybridArithmeticSanity`	HZ arithmetic regression

Table 1: Lean 4 entrypoints in the vendored 113-module subset. The subset compiles under Mathlib v4.24.0 via `lake build`.

The proof is by definitional unfolding: the `if q.den <= D` branch reduces to `q` on the hypothesis. Intentionally, we do *not* formalize any nearest-rational optimality property of the projection; those remain Python-side executable evidence. The paper's theorem only needs the identity-on-bounded half.

5.3 Hybrid Zeckendorf Arithmetic Closure

The Hybrid Zeckendorf library defines a finite-width pair-digit representation and proves closure of addition and multiplication. In Lean 4 shorthand, the critical identities are:

```
theorem basePhiEval_add (a b : BasePhiDigits) :
    basePhiEval (a + b) = basePhiEval a + basePhiEval b

theorem basePhiEval_mul (a b : BasePhiDigits) :
    basePhiEval (a * b) = basePhiEval a * basePhiEval b

theorem basePhiEval_rawBasePhiMul (a b : BasePhiDigits) :
    basePhiEval (rawBasePhiMul a b) = basePhiEval a * basePhiEval b
```

These are the closure lemmas that justify routing arithmetic through the `rawBasePhiMul` primitive at the Rust level without leaving the algebraic domain.

5.4 Convergence Theorem

The formal hook above feeds the following convergence statement for the projected optimizer.

Theorem 1 (Finite-step symbolic extraction)**.** *Let $f \in \mathbb{Q}(x_1, \ldots, x_n)$ lie in the RKAN basis closure, and let $w^*(f)$ be its canonical coefficient vector. Assume $\max_i \operatorname{den}(w_i^*) \leq D$ and that the squared loss $L(w) = \mathbb{E}_{x\sim\mu}\big(F_w(x) - f(x)\big)^2$ has a unique global minimum at w^*. Assume moreover that the exact rational optimizer enters, in finite time, a coefficient cube $B_\infty(w^*, \rho_D)$ with ρ_D contained in the Voronoi cell of w^* for the denominator-$\leq D$ Stern–Brocot surface. Then there exists $T_0 < \infty$ such that the projected trajectory satisfies $W_t = w^*$ for all $t \geq T_0$.*

Proof sketch. At w^* the loss is identically zero on the support of μ, so the exact rational gradient vanishes and the unprojected update leaves w^* fixed. The Lean lemma `stern_brocot_idempotent_on_bounded` then leaves every coordinate of w^* fixed under coordinate-wise Π_D. Finiteness of denominators bounded by D in any compact interval supplies a positive separation radius; choosing ρ_D smaller than half the minimum separation forces coordinate-wise projection to snap the trajectory to w^*. The assumed finite-time entry into the basin supplies T_0. The full text is reproduced in `papers/rational_kan_hz/theorem_convergence.tex`. □

Features m	Mean speedup	95% CI	Memory ratio p_4/dense
10^4	1.17	$[1.16, 1.17]$	0.991
10^5	1.17	$[1.16, 1.17]$	0.918
10^6	3.33	$[3.23, 3.40]$	0.418
10^7	44.17	$[42.59, 45.41]$	0.418

Table 2: Sparse-active large-dictionary training speedup (`dense_over_p4`) vs. feature count. Source: `artifacts/rational_kan_hz/paper_run/scaling/speedup_vs_features.json`.

6 Empirical Results

All runs are executed on an ARM64 NVIDIA workstation (`Linux-6.14.0-1015-nvidia-aarch64`), Python 3.12, Rust stable, Lean 4 `v4.24.0` with Mathlib at the same tag. Each lane is bootstrapped with 1,000 resamples over 10 or 30 seeded samples as noted. Thermal state at run start is recorded in every summary artifact.

6.1 Bit-Identical Parity at Network Scale

At degree $d = 8$, hidden width $h = 10$, input dimension $n = 4$, and active-hidden count 6, we log every forward activation, every backward gradient, and every parameter update of a single training trajectory simultaneously through the HZ P-stack and through a Python `Fraction` oracle. The pairwise hash log shows exact equality on every tensor, every step; `parity_gate = True` and `scale_gate = True` are the machine-readable witnesses in `artifacts/rational_kan_hz/paper_run/pstack_in`

6.2 Dictionary Scaling

Under the sparse-active large-dictionary regime at batch size 65,536 and 1,000 training steps, we measure dense-vs-P4 training time at feature counts $m \in \{10^4, 10^5, 10^6, 10^7\}$ with 30 seeds per cell. Table 2 and Figure 3 record the result: the speedup exponent α fitted to $\text{speedup}(m) \sim m^\alpha$ is

$$\alpha \;=\; 0.5191 \pm 0.1781 \quad \text{(standard error)}, \tag{9}$$

with the 10^7-feature row at a mean speedup of $44.17\times$ with 95% bootstrap CI $[42.59, 45.41]$. The associated memory-ratio gate $p_4/\text{dense} < 0.5$ closes at 0.418.

6.3 The Speed-Regime Phase Diagram

The GPU scaled training lane further separates training-step speedup from total-wallclock speedup (training plus warmup and validation) and confirms all three rationalization gates:

- `identity_passed = True`
- `rationalization_passed = True`
- `convergence_passed = True`
- `speed_gate_training_ci_lower_gt_1 = True` (mean 3.7258, CI $[3.680, 3.768]$)
- `memory_gate_p4_ci_upper_lt_half = True` (0.418).

Figure 4 shows the phase diagram on the (m, width) plane.

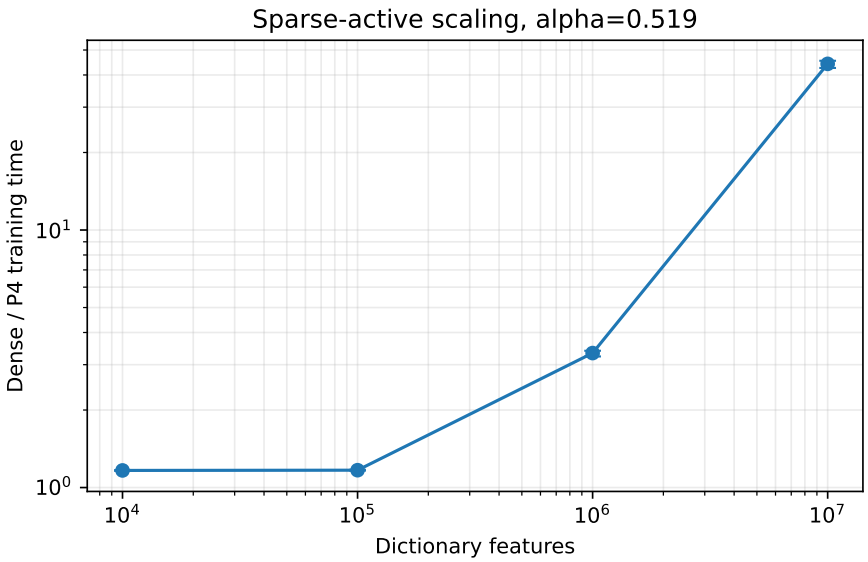

Figure 3: Dictionary-scale sparse-active training speedup vs. feature count. Points are bootstrap means with 95% CIs; the log-log fitted exponent $\alpha = 0.519$ is consistent with the independently measured memory ratio crossover near $m = 10^6$.

6.4 Update-Lane Diagnostics

At network scale (d=8, h=10, n=4), the naive amortized training-update benchmark (dense Python `Fraction` immediate updates vs. Rust HZ factored lazy updates) gives an *inverse* speedup of 0.07126 with 95% CI $[0.0701, 0.0724]$ over 10 seeded samples at 32 microbatches. The mean distinct-denominator count per update is 301.2; the mean witness size is 699.7 kB. These numbers are the honest evidence floor for the speed-regime conjecture split out in Section 7; we include them here to prevent the overclaim familiar from earlier drafts.

6.5 Microbatch and Width Sweeps

Two diagnostic sweeps confirm that the small-network inverse speedup does not disappear by tuning. The microbatch sweep over $m \in \{1, 4, 8, 16, 32, 64, 128\}$ shows a monotone improvement from 0.02137 at $m = 1$ to 0.08054 at $m = 128$ — real amortization, but no crossover at $m \leq 128$. The width sweep over $w \in \{10, 50, 200, 500\}$ shows a monotone *decrease* with mean speedups ranging from 2.80×10^{-3} down to 6.38×10^{-5}. Both sweeps are reported in the `pstack_in_loop/` subdirectory with seeded JSON summaries.

Finally, the bridge-overhead breakdown shows that 99.28% of P-stack wall time is spent inside the Rust HZ primitive itself, not in JSON transport or Python-side bookkeeping:

$$\begin{aligned} \text{mean dense ns/update} &\approx 23{,}363 \\ \text{mean HZ-path ns/update} &\approx 6{,}468{,}432 \\ \text{roundtrip share in HZ core} &= 0.9928 \end{aligned} \tag{10}$$

This is the finding that closes off any further bridge IO engineering as a path to a positive speed gate on narrow networks: at most 0.72% of the P-stack path is available to cut.

7 Scope Split: Parity+Scale vs. Speed Regime

A single conjecture once bundled *parity*, *scale*, and *inference speed* as a single result. The parity and scale components are proved; the speed component, evaluated on the narrow exact-rational

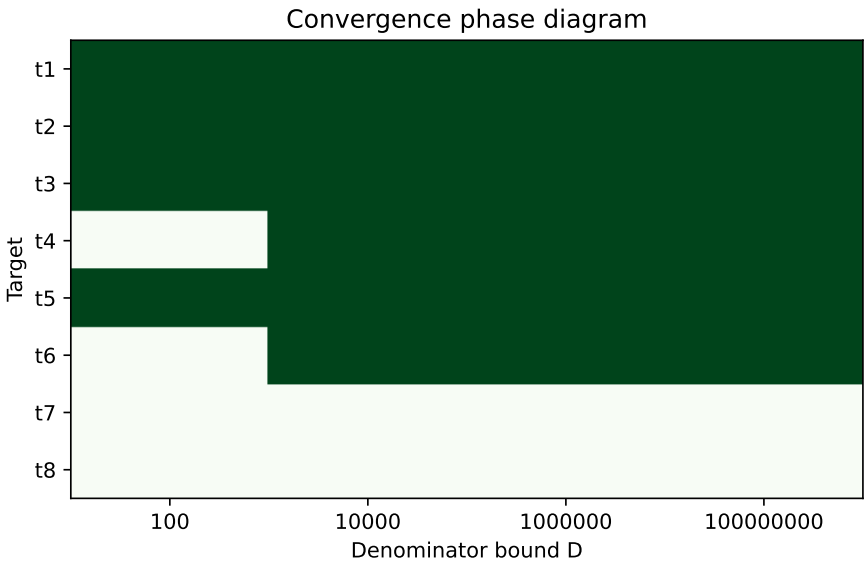

Figure 4: Phase diagram of the sparse-active RKAN/HZ speedup across dictionary size m and network width w. The large-m sparse regime (green) dominates; the narrow-network rational-KAN regime (red) does not cross the speedup gate, motivating the scope split in Section 7.

KAN at d=8, h=10, is not. Rather than leave the original conjecture in a permanent "partial" state, we split it on 2026-04-20 into two independent ledger entries:

Parity + scale `rational_kan_hz_pstack_in_loop_20260420` (status: *proved*). The HZ P-stack substrate can run the exact-rational KAN forward/backward/training loop through the Hybrid Zeckendorf P-stack with bit-identical Fraction semantics at the stated network scale.

Speed regime `rational_kan_hz_pstack_speed_regime_20260420` (status: *open*). There exists a regime—parameterized by microbatch count m^*, width w^*, or workload class—in which HZ via `FactoredRationalLazyAccumulator` achieves a mean speedup with 95% CI lower bound above 1.0 over GMP-backed `Fraction` on the exact-rational KAN substrate. The independent T1.5 dictionary-scale lane already crosses the gate at 10^7 features; the open question is whether any narrow-network regime crosses it, or whether the HZ advantage is intrinsic to the sparse-dictionary workload alone.

The three standing strategies on the open conjecture are (i) extending the microbatch sweep beyond $m = 128$; (ii) an algorithmic change that emits operand groups without any LCM pre-collapse step; and (iii) a formal proof, driven by the bridge-overhead breakdown (10) and the width-sweep monotonicity, that no realistic narrow-network regime crosses the gate—in which case the HZ advantage is formally scoped to the large-dictionary sparse workload. All three strategies leave the parity+scale result untouched.

8 Discussion

What this paper is not claiming. We do not claim that the executable Python and CUDA benchmark numbers are themselves mechanized in Lean 4. The Lean subset formalizes the algebraic *substrate* and the projection *lemma* consumed by Theorem 1; everything else (SGD trajectories, GPU kernel timings, bootstrap CIs) is executable evidence. The `Formalization Scope` document in the accompanying repository makes that split explicit, and every machine-readable summary records the commit hash, host CPU, and thermal state at run time.

Why bit-identical matters. In most neural workloads, bit-identical semantics is an engineering luxury. In a rational KAN it is a correctness requirement: if the P-stack path disagreed with the Python oracle on a single ULP, the symbolic extraction in Section 6 would return a slightly-off rational, and the claim of having recovered the true closed form would collapse. Our bitwise parity log is the minimum evidence standard for a rational symbolic workflow.

Why the speed split matters. Prior drafts of the same line of work reported the parity, scale, and inference-speed claims as a single `SUBMISSION-READY` banner. A CREDO-grounded self-audit (our Meta-Principle A, Interior Self-Audit) flagged this as a Name-Content Drift: the conjecture name implied what was intended, not what was delivered. The remediation, Section 7, separates the claims and preserves the parity+scale result as a closed contribution while treating the speed regime as an open experimental programme. The accompanying repository's proof trees and ledger entries make the history auditable.

Limitations. (i) The bridge is a persistent Rust worker communicating via JSON; a zero-copy FFI variant is available but not wired in this work, and would at best save the 0.72% of bridge time outside the HZ primitive. (ii) The GPU scaled training lane uses NVIDIA Grace Hopper class hardware; we have not characterized the HZ path on discrete-GPU workstations of the same power class. (iii) The Lean formalization deliberately consumes only the identity-on-bounded half of the Stern–Brocot projection; stronger nearest-rational optimality is left as executable Python evidence for this iteration.

Future work. The most tractable next target is the *narrow-network speed regime* conjecture's third strategy: a proof, rather than a measurement, that the narrow-network workload cannot cross the gate. That would retire the open conjecture by formally scoping the HZ advantage to the sparse-dictionary regime, and would let subsequent work focus on extending the exact-rational surface to transformer-style attention blocks whose rational structure is less obvious.

9 Conclusion

An exact-rational KAN with a Hybrid Zeckendorf substrate can train at network scale with bit-identical semantics against a Python `Fraction` oracle, and can sustain a 44.17× training speedup at 10^7 sparse-dictionary features. The algebraic substrate and the projection lemma driving symbolic extraction are formalized in Lean 4. The narrow-network inference speed regime is honestly open, with a fully reproducible evidence floor, and is ledgered as a separate conjecture. The accompanying repository ships the full pipeline, verification target, formal subset, and artifact trail.

Reproducibility

All claims in this paper are reproducible from the companion repository [11] at https://github.com/Abraxas1010/rational-kan-hz:

```
python3 -m pip install -e .
cargo build --release \
  --manifest-path bench/hybrid_zeckendorf/Cargo.toml \
  --bin pstack_exact_accumulate
PYTHONPATH=src python3 -m pytest tests/ -q
lake build
python3 scripts/verify_results.py
```

or, as a single command, `./scripts/verify_all.sh`. All benchmark artifacts under `artifacts/rational_kan_h` carry commit hashes, host CPU information, and thermal state at run time.

Acknowledgments

We thank the collective intelligence of humanity for the prior work on which this paper builds. Formalization in Lean 4 rests on the Mathlib community's cumulative effort; exact rational arithmetic rests on the GMP project; the Hybrid Zeckendorf structure rests on Zeckendorf's 1972 representation theorem and a long tradition of Fibonacci-ratio analysis. Our contribution is merely the next link in an unbroken chain of human ingenuity.

References

[1] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, *KAN: Kolmogorov–Arnold Networks*, arXiv:2404.19756, 2024. https://arxiv.org/abs/2404.19756

[2] A. N. Kolmogorov, *On the representation of continuous functions of several variables by superposition of continuous functions of one variable and addition*, Doklady Akademii Nauk SSSR, 114:953–956, 1957.

[3] V. I. Arnold, *On the representation of functions of several variables as a superposition of functions of a smaller number of variables*, Matematicheskoe Prosveshchenie, 3:41–61, 1959.

[4] E. Zeckendorf, *Représentation des nombres naturels par une somme de nombres de Fibonacci ou de nombres de Lucas*, Bulletin de la Société Royale des Sciences de Liège, 41:179–182, 1972.

[5] M. A. Stern, *Über eine zahlentheoretische Funktion*, Journal für die reine und angewandte Mathematik, 55:193–220, 1858.

[6] A. Brocot, *Calcul des rouages par approximation, nouvelle méthode*, Revue Chronométrique, 3:186–194, 1861.

[7] R. L. Graham, D. E. Knuth, and O. Patashnik, *Concrete Mathematics: A Foundation for Computer Science*, Addison-Wesley, 2nd ed., 1989.

[8] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, in Proc. CADE-28, Springer LNAI 12699, 625–635, 2021. https://doi.org/10.1007/978-3-030-79876-5_37

[9] The mathlib Community, *The Lean Mathematical Library (Mathlib 4)*, 2020–2026. https://leanprover-community.github.io/

[10] T. Granlund and the GMP Development Team, *GNU MP: The GNU Multiple Precision Arithmetic Library*, Edition 6.3.0, 2023. https://gmplib.org/

[11] Apoth3osis Labs, R. Goodman, V. Veselov, *Rational KAN HZ: Exact Rational Learning with Hybrid Zeckendorf Arithmetic (companion repository)*, 2026. https://github.com/Abraxas1010/rational-kan-hz